

A Step-by-Step Introduction to Error-Correcting Codes: From Hamming to Reed–Solomon

Pedro Cosme da Costa Vieira (pedro.cosme.vieira@gmail.com)

PENTAGON, Portugal

April 14, 2026

Abstract: Reliable storage and transmission of digital information require mechanisms that mitigate noise and physical imperfections. Direct approaches based on hardware robustness or full data replication are inefficient at scale, making error-correcting codes essential in modern systems. This work introduces Hamming and Reed–Solomon codes as two fundamental redundancy schemes. Hamming codes enable single-bit error correction in small blocks with minimal overhead, while Reed–Solomon codes extend error protection to multiple-symbol recovery across larger data structures. Reed–Solomon coding is first presented using a simplified integer-based model to emphasize its conceptual structure through redundancy and systems of equations, and is then contrasted with finite-field implementations used in practice. These methods underpin modern storage and communication technologies, enabling high-density memory systems and reliable high-speed data transmission.

Keywords: Error-correcting codes, Hamming codes, Reed–Solomon, finite fields, data redundancy, digital communications.

The writing of this text received substantial assistance from ChatGPT 5.

1. Introduction

In any real-world system—whether storing data on a physical medium or transmitting it across a distance—information is never perfectly preserved. Errors, noise, and distortions are not exceptions but the rule. Bits may flip due to radiation or hardware imperfections, signals may degrade over distance, and interference can alter or erase parts of the message. Despite these challenges, communication systems are designed to remain robust, allowing meaning to be recovered even when the underlying data is partially corrupted.

A simple example illustrates this resilience. Consider the following corrupted text:

This is a simple txt whre ltrrs are mssng and sme are flppd, bt yu can still undrstand the msg becace the strcture of wrds and sentnces is famliar.

Even though several letters are missing or altered, the message remains understandable. This is because human language contains redundancy, and the number of plausible word and sentence structures is limited. The same principle underlies many technical solutions to noise: by introducing controlled redundancy and structure, systems can detect and often correct errors.

The redundancy present in natural language is precisely what allows messages to be compressed using techniques such as ZIP encoding. By removing predictable patterns and repeated structures, compression reduces the size of the message. However, this also makes the compressed data far more sensitive to errors: a single bit flip can have a much larger impact than in the original text, potentially corrupting an entire segment of the message.

A historical parallel can be found in the writing of the Old Testament in ancient Hebrew, where vowels were often omitted to save space. In this condensed form of writing, the reader relies heavily on context to reconstruct the intended words. However, this also means that an error or omission of a single consonant can significantly increase the ambiguity, sometimes making correct interpretation difficult or even impossible. For example, the original phrase “We live in peace” can be reduced to “W lv n pc”. This reduced form can be interpreted as “We live in peace” or “We love no pace”. Now, if there is an error in the last letter, yielding “W lv n pt”, the original sentence can no longer be reliably reconstructed, and it may instead be read as “We live in pit”.

Since all information is inherently affected by noise, omissions, and distortions, it becomes necessary to ensure reliable recovery despite these imperfections. Over time, a wide range of methodologies has been developed to detect and correct errors in data storage, reading, writing, and transmission. Some of these methods are simple and intuitive, while others are highly sophisticated and mathematically complex.

Regardless of the specific methodology employed, all error detection and correction techniques share a fundamental principle: the deliberate introduction of redundancy. By adding extra information beyond the minimum required to represent the message, systems gain the ability to identify inconsistencies and, in many cases, to reconstruct the original data.

In the following section, we begin with the simplest possible approach: replicating the same information multiple times. By storing or transmitting an odd number of identical copies and applying a majority consensus rule, it becomes possible to recover the correct message even when some of the copies are corrupted.

2. Hamming codes for bit correction

We begin again with the sentence “We live in peace.” The simplest and most intuitive approach to error correction is to introduce redundancy by repeating the information. For example, the sentence can be expanded as:

“WWWeee llliiivvveee iinnn pppeeeaaaccceee”.

In this representation, each character is repeated multiple times. Even if some letters are altered during storage or transmission, such as:

“WzWeae i llliiivaveee iipnnn pspeeeavacceue”,

the original message can still be recovered by applying a majority consensus rule to each group of repeated characters. As long as most of the copies remain correct, the intended letter can be inferred reliably.

We now consider a binary representation of information. For example, a hard disk drive (HDD) stores data in sectors of 4096 bytes. Assuming a bit-flip error probability of approximately one in ten millions per bit, the probability of at least one error occurring within a sector becomes significant. Under these conditions, one would expect roughly one corrupted sector every 306 sectors, which is clearly unacceptable for reliable storage.

```
import math
def sector_error_probability(n, sector_bytes, bit_error_rate):
    "Compute probability of exactly n bit errors in a sector, Binomial distribution."
    N = sector_bytes * 8 # total bits in sector
    if n < 0 or n > N:
        return 0.0
    # binomial probability
    return math.comb(n, k) * (p**k) * ((1 - p)**(n - k))

print(f"Probability of error: 1/{round(1/(1-sector_error_probability(0,4096,1e-7)),1)}
sectores")
```

Probability of error: 1/305.7 sectores

A straightforward way to mitigate this problem is repetition coding. Each sector is stored three times, and the final value is recovered by majority voting across the copies. While this approach significantly improves robustness against errors, it comes at a high cost: the effective storage capacity of the HDD is reduced to one third of its original value.

Parity bits

A next step in error control is the use of parity bits. Unlike repetition codes, parity does not allow error correction; instead, it enables error detection.

In its simplest form, a single parity bit is added to a block of data to ensure that the total number of 1s is either even (even parity) or odd (odd parity). This allows the detection of any odd number of bit flips within the block. However, even numbers of errors may continue to go undetected.

```
def undetected_error(sector_bytes, bit_error_rate):
    undetected=0
    for n in range(2,100,2):
        undetected +=sector_error_probability(n,sector_bytes,bit_error_rate)
    return undetected

non_detected =undetected_error(4096,1e-7)
print(f"Probability of error not detected {non_detected:0.6%}")
```

Probability of error not detected 0.000535%

We can image a multi-bit checksum. For example, if a 4-bit checksum were used (similar to a hash function), there are 16 possible output states. In an ideal deterministic interpretation, one could think that up to 15 different corrupted patterns might map to different detected states. In practice, however, detection is not

uniform across all error patterns. While many multi-bit errors will be detected, there is no strict guarantee, and the effective reliability depends on the structure of the encoding. As a result, the practice is to divide the block and to use several independent parity checks.

Once an error is detected in a sector, an alternative copy can be used. If two copies are stored, the probability that both are corrupted simultaneously is significantly lower, greatly improving reliability while maintaining relatively high storage efficiency compared to full repetition coding.

The Hamming Construction – From Detection to Correction

Hamming codes represent a major step forward in error control, as they allow both the detection and correction of single-bit errors without the need to duplicate the entire message. Instead of repeating information, they introduce carefully placed parity bits that encode structural relationships across the data.

This means that, unlike repetition schemes where redundancy grows linearly with robustness, Hamming codes achieve error correction with significantly lower overhead.

The foundations of this approach were established by Hamming(1950), during his work at Bell Labs, where he developed the first practical error-correcting code capable of detecting and correcting single-bit errors. This work marked a fundamental milestone in the theory of reliable digital communication and storage systems.

To illustrate the idea, consider a block of 64 data bits. Instead of duplicating the full block (which would require $3 \times 64 = 196$ bits), a Hamming code adds only a small number of parity bits. For 64 data bits, 7 parity bits are sufficient to locate and correct any single-bit error within the 71-bit encoded block.

In practice, ECC memory systems are based on extended Hamming codes. A typical implementation protects 64 bits of data using 8 additional parity bits, resulting in a 72-bit memory word. This overhead allows the system to correct any single-bit error and detect (but not correct) double-bit errors. This is why ECC memory modules are wider (9 chips) than standard memory (8 chips): instead of storing exactly 64 bits per word, they store 72 bits, embedding redundancy directly into the hardware.

To illustrate the power of Hamming codes, consider a single data symbol represented by 64 bits. In this example, the symbol can be viewed as an unsigned integer, which is then converted into its binary representation.

```
def int_to_bits(n):
    return list(map(int, bin(n)[2:]))

def bits_to_int(bits):
    return int("".join(map(str, bits)), 2)

number = 12345678901234567890
data = int_to_bits(number)
print("data:", "".join(map(str, data)))

data: 1010101101010100101010011000110011101011000111110000101011010010
```

Instead of transmitting or storing this 64-bit word directly, we construct a Hamming-encoded version of the data.

To correct at most one error in a symbol of m bits, it is necessary to introduce r parity bits such that the condition is satisfied.

$$2^r \geq m+r+1.$$

This ensures that every possible single-bit error (including the no-error case) can be uniquely identified by the syndrome.

By adding an additional global parity bit, it becomes possible to detect the occurrence of multiple-bit errors. The global parity distinguishes between even and odd numbers of bit flips, allowing the receiver to identify when a non-zero syndrome is not due to a single-bit error but to multiple errors, thus preventing incorrect correction (Extended Hamming - SECDED).

Data	1	0	0	1	0	1	1	1	0	0	1	0	1	0	0	0						Parity	
Coded	0	0	1	0	0	1	0	0	1	1	1	0	0	1	0	0	1	0	0	0			
bit1	0		1		0		1		0		1		0		1		0		0	0	0		
bit2		0	1			0	1			1	1			0	1			1	0				
bit3				0	0	0	1					1	0	0	1				0	0	1		
bit4								0	0	1	1	1	0	0	1						0		
bit5																0	0	1	0	0	0		
Coded	0	0	1	1	0	0	1	0	0	1	1	1	0	0	1	1	0	1	0	0	0	bit6	1

Table 1 – 16+5 bit Hamming Code example

For a 64-bit symbol, this condition requires $r = 7$ parity bits, since $2^6 \geq 64+6+1$ is false while $2^7 \geq 64+7+1$ is true. Similarly, for a 4096-bit symbol, at least $r=13$ parity bits are required.

```
# number of parity bits needed
r = 0
while (2 ** r) < (m + r + 1):
    r += 1
total_length = m + r
```

The parity bits are placed at positions corresponding to powers of two (1, 2, 4, 8, 16, 32, 64, ...), and the original data bits are inserted into the remaining positions of the encoded word.

```
code = []
# place data and parity placeholders
for i in range(1, total_length + 1):
    if math.log2(i).is_integer():
        code.append(0) # parity placeholder
    else:
        code.append(data_bits[j])
```

This process effectively interleaves redundancy with the original message, expanding the symbol into a longer structured codeword (we represent them as x but we have use 0).

codedword: $xx1x010x1011010x101001010100110x00110011101011000111110x000101011010010$

Once the structure of the Hamming code is defined, the next step is to determine the value of each parity bit. Each parity bit is responsible for enforcing consistency over a specific subset of positions in the codeword.

The key idea is that every position in the codeword can be represented in binary form. A parity bit placed at position 2^k checks all bit positions whose binary representation contains a 1 in the k -th bit. Being so, each parity bit selects positions in the codeword according to a binary pattern, effectively partitioning the data space into overlapping subsets defined by powers of two.

During encoding, the parity bit positions are initially filled with zeros and treated as part of the full codeword. The parity values are then computed over this complete structure, including both data and placeholder parity bits (a total of $m+r$ bits).

Parity bit at position 1 checks all positions whose binary index has the least significant bit set to 1:

0000**1**, 0001**1**, 0010**1**, 0011**1**, 0100**1**, 0101**1**, 0110**1**, 0111**1**, 1000**1**, ...
= 1, 3, 5, 7, 9, 11, 13, 15, 17, ...

Parity bit at position 2 checks all positions whose binary index has the second least significant bit set to 1:

000**1**0, 000**1**1, 001**1**0, 001**1**1, 010**1**0, 010**1**1, 011**1**0, 011**1**1, 100**1**0, ...
= 2, 3, 6, 7, 10, 11, 14, 15, 18, ...

Parity bit at position 4 checks all positions whose binary index has the third least significant bit set to 1:

00**1**00, 00**1**01, 00**1**10, 00**1**11, 01**1**00, 01**1**01, 01**1**10, 01**1**11, 10**1**00, ...
= 4, 5, 6, 7, 12, 13, 14, 15, 20, ...

Finally, the placeholders are replaced with the computed parity values, resulting in a fully consistent codeword.

```
def build_hamming_code(data_bits):
    "Insert parity bits at positions 1,2,4,8,..."
    m = len(data_bits)
    # number of parity bits needed
    r = 0
    while (2 ** r) < (m + r + 1):
        r += 1
    total_length = m + r
    code = []
    j = 0
    # place data and parity placeholders
    for i in range(1, total_length + 1):
        if math.log2(i).is_integer():
            code.append(0) # parity placeholder
        else:
            code.append(data_bits[j])
            j += 1
    # compute parity bits
    for i in range(r):
        p = 2 ** i # position of the parity bit (power of two index)
        parity = 0 # initial value of the parity bit
        for j in range(1, total_length + 1): # all positions in the codeword
```

```

        # check whether position j contributes to this parity bit
        if j & p: # bitwise mask: checks if the p-th bit is set in j
            parity ^= code[j - 1] # compute XOR accumulation
        code[p - 1] = parity # store computed parity bit
    return code

```

```

number = 12345678901234567890
data = int_to_bits(number)
codeword = build_hamming_code(data)
print("".join(map(str, codeword)), len(codeword))

```

```

xx1x010x1011010x101001010100110x00110011101011000111110x000101011010010
011001001011010111010010101001100011001100111110000101011010010

```

After transmission or storage, the received codeword is checked by recomputing the parity constraints. This process produces a quantity known as the syndrome.

The syndrome is computed using the same parity-check structure applied during encoding. Each parity check is evaluated again over the received word, and the results are combined into a binary number.

If the syndrome is equal to zero, all parity constraints are satisfied, indicating that no error has been detected. To illustrate this, consider the example of position 5:

```

Initially    0101001010100110 -> 7 1s -> 1
Coded       1101001010100110 -> 8 1s -> 0 -> correct

```

During encoding, the parity bit is chosen so that the overall XOR of the corresponding group is zero. In this example, the parity calculation initially yields a value of 1, so the parity bit is set to 1 instead of 0 in order to enforce the constraint.

When the syndrome is later computed, the same parity check is performed again. If no error has occurred, the recomputed XOR must return to zero, confirming that the encoded structure remains consistent. In this sense, the process behaves like a comparison between the expected and actual parity, but it is performed implicitly through the parity constraints.

If the syndrome is non-zero, its binary value directly identifies the position of a corrupted bit within the codeword. In the previous example, if an error affects bit position 5, the violated parity checks combine to produce a syndrome equal to 5, allowing the receiver to locate and correct the error. This remarkable property arises from the structured way in which parity bits partition the codeword: each bit position produces a unique pattern of parity violations when it is flipped. As a result, the syndrome acts as a coordinate system that maps error patterns to specific locations.

If more than one bit is corrupted, for example errors occurring at positions 3 and 6, the syndrome remains non-zero, indicating that an inconsistency has been detected. In this case, the combined effect of multiple errors produces a pattern of parity violations that no longer corresponds to a valid single-bit position. As a result, the error cannot be correctly located or corrected.

```

def compute_syndrome(codeword):
    n = len(codeword)
    r = int(math.log2(n)) + 1
    syndrome = 0
    for i in range(r):
        # compute parity bits

```

```

p = 2 ** i
parity = 0
for j in range(1, n + 1):
    if j & p:
        parity ^= codeword[j - 1]
# compute parity bits
if parity == 1: #error location
    syndrome += p
return syndrome

```

When the syndrome is, for example, 37, it means that parity has an error in the positions 1, 3 and 6, then the syndrome becomes $1^0 + 2^2 + 2^5 = 37$

Error->	0	0	1	1	0	0	1	0	0	1	1	0	0	0	1	1	0	1	0	0	0	0	bit6	0	p
bit1	0		1		0		1		0		1		0		1		0		0		0		0	0	
bit2		0	1			0	1			1	1			0	1			1	0				0	0	
bit3				1	0	0	1					0	0	0	1						0	0	1	4	
bit4								0	0	1	1	0	0	0	1								1	8	
bit5																1	0	1	0	0	0	0	0		
Syndrome																							12		

Table 2 – Example from Table 1 with an error in the position 12

Once the position of the error has been identified through the syndrome, correction becomes straightforward. The receiver simply flips the bit at the indicated position, restoring the original codeword.

After correcting a potential error, the original data symbol can be recovered by removing the parity bits from the codeword. Since parity bits are located at positions corresponding to powers of two, the decoding process simply consists of extracting all remaining positions.

```

def correct_extract(codeword, pos):
    #correct codeword
    corrected = codeword[:]
    if pos != 0:
        corrected[pos - 1] ^= 1
    #extract data from corrected codeword
    data = []
    for i in range(1, len(corrected) + 1):
        # check if i is NOT a power of two
        if not math.log2(i).is_integer():
            data.append(corrected[i - 1])
    return data

```

This step completes the full encoding-decoding cycle: the data is first expanded with structured redundancy, then verified and corrected if necessary, and finally reduced back to its original form.

```

number = 12345678901234567890123456789012345678901234567890
data = int_to_bits(number)
codeword = build_hamming_code(data)
corrupted = codeword[:]
corrupted[123] = not(corrupted[123])

syndrome = compute_syndrome(corrupted)
corrected = bits_to_int(correct_extract(corrupted, syndrome))
print("Detected position:", syndrome)
print("Corrected:", corrected)

```



```
print("Success:", number==corrected)
```

Detected position: 124

Corrected: 12345678901234567890123456789012345678901234567890

Success: True

This completes the fundamental mechanism of Hamming codes: errors are not only detected, but also precisely located and corrected with minimal redundancy.

Let us return to the previous example of a storage system with 64 KB sectors, where each bit has an independent error probability of $1\text{E-}7$. The probability of each sector being corrupted is 5.1% and 3 copies would reduce this probability to 0.35% (with a reduction of 67% in capacity).

If the sectors are divided into symbols of 8 bytes (64 bits), and each symbol is protected using a Hamming code, there is almost 40000 times reduction in the probability of having a corrupted sector, with just 10% reduction in capacity:

```
symbol=8 #bytes
error = 0
for i in range(2,100):
    error += sector_error_probability(i,symbol,1e-7)

error_sectorh = 1-sector_error_probability(0,64*1024//symbol, error)
print("ERROR sector hamming:", error_sectorh)
```

ERROR sector hamming: 1.3211966146675636e-06

While Hamming codes are highly effective at detecting and correcting single-bit errors within a block, their capabilities become limited when multiple errors occur within the same block. In such cases, the pattern of errors no longer corresponds to a valid syndrome, and reliable correction is no longer possible.

This limitation becomes particularly relevant in practical systems where information is transmitted or stored in larger units, such as packets or sectors. For example, data may be divided into packets with a payload of 512 bytes. In some scenarios, errors do not occur as isolated bit flips, but rather as the loss or corruption of entire packets.

Within each packet, Hamming codes can still be used to correct small, random errors. However, they are not suitable for recovering a completely lost or severely corrupted block of data.

It is in this context that Reed–Solomon codes become essential. Instead of operating at the bit level, these codes work on larger symbols and are specifically designed to recover missing or corrupted blocks. By introducing redundancy across multiple packets, Reed–Solomon codes allow the reconstruction of entire data units, even when some packets are lost entirely.

This capability makes them particularly well suited for communication systems, storage devices, and distributed data systems, where burst errors or packet loss are common.

If Hamming codes provide precision at the bit level, Reed–Solomon codes provide robustness at the block level.

3. Reed–Solomon Codes for Block-Level Error Correction

Consider a message of 16,384 bytes to be transmitted over a noisy channel. The noise may manifest as independent bit flips or as the loss of entire portions of the message.

The message is divided into 32 packets, each containing 512 bytes of payload. Each packet is assigned an index, represented for simplicity by an 8-byte word, increasing the packet size to 520 bytes.

The first level of error correction consists of dividing each packet into smaller words and applying Hamming codes at the bit level.

i) The packet is partitioned into words. The choice of word size is critical: it must be small enough to ensure that the probability of multiple bit errors within the same word remains sufficiently low, so that Hamming correction is effective. At the same time, it should not be too small, in order to avoid excessive redundancy.

ii) Each word is encoded using Hamming codes. Assuming a word size of 8 bytes (64 bits), each encoded word becomes 9 bytes. For a packet composed of $520/8=65$ such words, the total packet size becomes:

$$65 \times 9 = 585 \text{ bytes}$$

3.1. Reed – Solomon Codes Using Integers

While Hamming codes protect against random bit errors within each packet, they do not address the loss of entire packets. Reed–Solomon (1960) codes overcome this limitation by introducing redundancy across multiple packets, enabling recovery from packet loss.

In order to apply Reed–Solomon coding, it is necessary to preserve the correct ordering of packets and to identify which packets are missing or corrupted. This allows the decoder to treat missing packets as erasures, a condition under which Reed–Solomon codes are particularly effective.

By adding a number of redundant packets to the original set, the system can tolerate the loss of several packets. As long as the number of missing packets does not exceed the redundancy introduced, the original data can be fully reconstructed.

In this way, Reed–Solomon codes complement Hamming codes: while Hamming ensures local consistency within each packet, Reed–Solomon provides global resilience across packets, allowing recovery from complete packet loss. This combination of local and global redundancy forms the basis of many modern storage and communication systems.

At this stage, we consider again that the message is divided in 32 packets, each with a size of 585 bytes.

In Reed–Solomon terminology, each byte is treated as a symbol. Then, using 8-bit symbols, 0 to 255, each packet becomes a sequence of 585 symbols.

For each byte position, the system collects the 32 corresponding symbols (homologous), one from each packet. These symbols are interpreted as elements of a finite field, typically $GF(2^8)$.

Reed–Solomon encoding is then applied to this set of symbols, generating additional redundancy symbols that will form the redundant information, $P(x)$, with x equal to 1, 2, ...

$$p(x) = b_1 + b_2x + b_3x^2 + \dots$$

Although P 's are also in the finite field $GF(2^8)$, to increase clarity of the concepts, in this simplified model, redundant symbols are allowed to exceed 8-bit, which removes the constraints of modular arithmetic while preserving the structure of the coding scheme.

```
def P_RScode(x,slice):
    result=0
    for i,symbol in enumerate(slice):
        result += symbol*x**i
    return result

slice = [95, 85, 103, 36, 133, 175, 203, 129, 33, 29, 50, 127, 29, 217, 149, 227, 241,
         125, 213, 115, 212, 191, 226, 246, 168, 57, 172, 128, 64, 169, 236, 126]
slice_coded = slice[:]
for i in range(1,4):
    slice_coded.append(P_RScode(i,slice))
print(slice_coded)

[95, 85, 103, 36, 133, 175, 203, 129, 33, 29, 50, 127, 29, 217, 149, 227, 241, 125, 213,
115, 212, 191, 226, 246, 168, 57, 172, 128, 64, 169, 236, 126, 4509, 669145653301,
141022060133913305]
```

Single Erasure Recovery: Subtraction

A particularly simple and intuitive case of block-level recovery occurs when only a single packet is lost. This situation is the “trivial” case of Reed–Solomon coding, where the redundancy corresponds to evaluating the polynomial at a single point at $x = 1$.

$$p_1 = b_1 + b_2 + b_3 + b_4 + \dots$$

Assuming we lost the symbol b_3 , we compute it by summing:

$$b_3 = p_1 - (b_1 + b_2 + b_4 + \dots)$$

In this simplified setting, the parity block is obtained by combining all data blocks through a linear operation, effectively forming one equation that relates all variables.

```
#Lost the packet 11
slice_erased = [95, 85, 103, 36, 133, 175, 203, 129, 33, 29, "x", 127, 29, 217, 149,
                227, 241, 125, 213, 115, 212, 191, 226, 246, 168, 57, 172, 128, 64,
                169, 236, 126, 4509, 669145653301, 141022060133913305]
slice_erased[10] = -slice_erased[-3]
slice_erased[10] = -1*P_RScode(1,slice_erased[:-3]) #Recover symbol at position 11
print(slice_erased)
[95, 85, 103, 36, 133, 175, 203, 129, 33, 29, 50, 127, 29, 217, 149, 227, 241, 125, 213,
115, 212, 191, 226, 246, 168, 57, 172, 128, 64, 169, 236, 126, 4509, 669145653301,
141022060133913305]
```

Transmission	B7	B3	B9	B2	...	
Redundancy	B1	B2	B3	B4	...	-> P1
Correction	B1	-P1	B3	B4	...	-> -B2

Figure 1 – Transmission out-of-order; Computation of the redundancy; and Correction of B2 erasure

Multiple Erasures Recovery : Solving Vandermonde System of Equations

Considering the polynomial $p(x) = b_1 + b_2*x + b_3*x^2 + \dots$, the redundancy symbols correspond to evaluations of this polynomial at different points, such as $p(1)$, $p(2)$, $p(3)$. When some symbols are missing, their contributions can be isolated by subtracting the known terms from each equation. This leads to a linear system where the unknowns correspond to the missing symbols, and the coefficients are powers of the evaluation points. The resulting system has a Vandermonde structure and can be solved using standard linear algebra techniques.

Assume that three symbols are missing from the sequence, for example at positions 7, 18 and 25. Let these unknown values be denoted by x_7 , x_{18} and x_{25} and that three redundancy symbols have been computed, corresponding to the evaluations $p(1)$, $p(2)$, and $p(3)$. Each of these provides one independent equation.

After subtracting the contribution of all known symbols, the system can be written as:

$$\begin{aligned} x_7 + x_{18} + x_{25} &= S_1 \\ 2^6*x_7 + 2^{17}*x_{18} + 2^{24}*x_{25} &= S_2 \\ 3^6*x_7 + 3^{17}*x_{18} + 3^{24}*x_{25} &= S_3, \end{aligned}$$

where S_1 , S_2 , S_3 are the residual values obtained after subtracting the known symbol contributions from $p(1)$, $p(2)$, $p(3)$, respectively. This system can be written in matrix form as:

$$\begin{vmatrix} 1 & 1 & 1 \\ 2^6 & 2^{17} & 2^{24} \\ 3^6 & 3^{17} & 3^{24} \end{vmatrix} * \begin{vmatrix} x_7 \\ x_{18} \\ x_{25} \end{vmatrix} = \begin{vmatrix} S_1 \\ S_2 \\ S_3 \end{vmatrix} \Leftrightarrow \begin{vmatrix} x_7 \\ x_{18} \\ x_{25} \end{vmatrix} = \text{Inv} \begin{vmatrix} 1 & 1 & 1 \\ 2^6 & 2^{17} & 2^{24} \\ 3^6 & 3^{17} & 3^{24} \end{vmatrix} * \begin{vmatrix} S_1 \\ S_2 \\ S_3 \end{vmatrix}$$

Since the number of equations matches the number of unknowns, and the matrix has a Vandermonde structure with distinct evaluation points, the system admits a unique solution. Therefore, all missing symbols can be exactly recovered. This illustrates a key property of Reed–Solomon codes: as long as the number of erasures does not exceed the number of redundancy symbols, full recovery is guaranteed.

```
from sympy import Matrix
```

```
def recover_rs_integer_exact(slice_erased):
    "Exact recovery using integer arithmetic (no floating point)."
    k = sum(1 for v in slice_erased if v == 'x')
    data = slice_erased[:-k]
    parity = slice_erased[-k:]
    missing_idx = [i for i, v in enumerate(data) if v == 'x']
    known = [(i, v) for i, v in enumerate(data) if v != 'x']
    A = []
    b = []
```

```

for eq in range(k):
    x_val = eq + 1
    rhs = parity[eq]
    for idx, val in known:
        rhs -= val * (x_val ** idx)
    row = []
    for idx in missing_idx:
        row.append(x_val ** idx)
    A.append(row)
    b.append(rhs)
# exact solve (no floats)
A = Matrix(A)
b = Matrix(b)
solution = A.LUsolve(b)
recovered = data[:]
for idx, val in zip(missing_idx, solution):
    recovered[idx] = int(val)
return recovered

slice_erased = [95, 85, 103, 36, 133, 175, 'x', 129, 33, 29, 50, 127, 29, 217, 149,
                227, 241, 'x', 213, 115, 212, 191, 226, 246, 'x', 57, 172, 128, 64,
                169, 236, 126, 4509, 669145653301, 141022060133913305]

recovered = recover_rs_integer_exact(slice_erased)

print("Recovered data:", print(recovered))

Recovered data: [95, 85, 103, 36, 133, 175, 203, 129, 33, 29, 50, 127, 29, 217, 149, 227, 241, 125, 213, 115,
212, 191, 226, 246, 168, 57, 172, 128, 64, 169, 236, 126]

```

When missing symbols exceed the number of available redundancy symbols, recovery is impossible.

3.2. Finite Field Implementation (Brief Note)

The integer-based formulation presented above provides a clear and intuitive understanding of the Reed–Solomon mechanism, as it relies on standard arithmetic and leads to a classical system of linear equations. However, it has a significant practical limitation: the values involved grow very rapidly, leading to large integers and potential numerical instability.

In practical implementations, Reed–Solomon codes are defined over finite fields, typically $GF(2^8)$. In this setting, each symbol remains bounded to 8 bits, regardless of the number of operations performed. This ensures that all encoded and decoded values fit within a fixed-size representation, which is essential for efficient storage and transmission.

The trade-off is that arithmetic in finite fields is less intuitive than standard integer arithmetic. Operations such as addition, subtraction, multiplication, and division are no longer the usual ones, but are instead defined according to the algebraic structure of the field. In particular, addition corresponds to the bitwise XOR operation, while multiplication and division follow polynomial arithmetic modulo an irreducible polynomial.

The computation of the redundancy symbols $P(1)$, $P(2)$, ... follows the same conceptual structure as in the integer case. However, all operations involved — including those used to build and solve the system of equations for recovery — must be carried out using the finite field operations.

```
def gf_add2(a, b):
    "Sum / subtract two elements in GF(2^8)"
    return a ^ b #XOR

def gf_mul2(a, b):
    "Multiply two elements in GF(2^8) using the AES polynomial (0x11b)"
    result = 0
    for _ in range(8):
        if b & 1:
            result ^= a # add (XOR)
        carry = a & 0x80 # check if highest bit is set
        a <<= 1 # multiply by x
        if carry:
            a ^= 0x11b # reduce modulo irreducible polynomial
        a &= 0xFF # keep 8 bits
        b >>= 1
    return result

def gf_inv(a):
    "Inverse in GF(2^8) using brute-force (educational)."
    for i in range(1, 256):
        if gf_mul2(a, i) == 1:
            return i
    raise ValueError("No inverse found")

def RScode_P(x, bs):
    "Evaluate polynomial P(x) = b0 + b1*x + b2*x^2 + ... over GF(2^8)"
    result = bs[0]
    power = 1 # x^0
    for b in bs[1:]:
        power = gf_mul2(power, x) # compute next power of x
        result = gf_add2(result, gf_mul2(b, power)) # add b_k * x^k
    return result

slice = [95, 85, 103, 36, 133, 175, 203, 129, 33, 29, 50, 127, 29, 217, 149, 227, 241,
         125, 213, 115, 212, 191, 226, 246, 168, 57, 172, 128, 64, 169, 236, 126]
print([RScode_P(i,slice) for i in range(10)])

[95, 121, 204, 103, 179, 232, 244, 208, 182, 167]
```

3.3. Application: RAID5 - Redundant Array of Independent/Inexpensive Disks

In applications where data loss is unacceptable, a cost-effective and widely used solution is the implementation of RAID5 (Redundant Array of Independent Disks) systems. Instead of relying on a single storage device, data is distributed across multiple hard drives, with additional redundancy to ensure recoverability in case of failure.

A common configuration is the so-called N+1 scheme, where N disks store data and one additional disk stores redundancy information. For example, a system requiring 100 TB of storage may use five data disks of

20 TB each, combined with one additional 20 TB disk dedicated to redundancy. This results in a total of 120 TB of physical storage, with a redundancy overhead of 20%.

In a simplified model, the redundancy disk stores the XOR of corresponding symbols (or blocks) across the N data disks. That is, for each position, the parity value is computed as:

$$D_6 = D_1 + D_2 + D_3 + D_4 + D_5 = D_1 \text{ XOR } D_2 \text{ XOR } D_3 \text{ XOR } D_4 \text{ XOR } D_5$$

Where D_i represents the data stored in each disk. If any single disk fails, its contents can be reconstructed by recomputing the XOR of the remaining disks and the parity disk. For example, the D_4 fails:

$$D_4 = D_6 - (D_1 + D_2 + D_3 + D_5) = D_6 \text{ XOR } D_1 \text{ XOR } D_2 \text{ XOR } D_3 \text{ XOR } D_5$$

This mechanism is the “trivial” case of a Reed–Solomon-like scheme with a single redundancy symbol, P_1 , where recovery is achieved through a simple linear relation. While more advanced RAID5 levels extend this concept to tolerate multiple simultaneous failures, the basic XOR-based approach already provides an efficient and practical solution for single-disk fault tolerance.

```
#data
d1, d2, d3, d4, d5 = 33, 50, 127, 29, 217
#redundancy
d6 = d1 ^ d2 ^ d3 ^ d4 ^ d5
#d4 failed - Recovery
d4_r = d1 ^ d2 ^ d3 ^ d5 ^ d6
print(f"verification: {d4} = {d4_r}")
```

verification: 29 = 29

The effectiveness of RAID5 systems can be quantified using real-world reliability data. According to statistics published by Backblaze (2025), during the fourth quarter of 2025, 943 failures were observed among 337,192 hard drives, corresponding to a failure probability of approximately 2.8×10^{-3} per drive per quarter.

In a RAID5 system, a single disk failure does not result in data loss, as the missing information can be reconstructed from the remaining disks and the redundancy data. However, this recovery process requires reading all disks in the array, which takes a non-negligible amount of time.

Assuming a rebuild bandwidth of 250 MB/s and that only 10% of system time is allocated to recovery operations, rebuilding a 20 TB disk requires approximately 10 days. During this period, the system remains vulnerable: a second disk failure may lead to irreversible data loss. Under these assumptions, a RAID5 (5+1) configuration (five data disks and one parity disk) reduces the probability of unrecoverable data loss by approximately two orders of magnitude, from 2.8×10^{-3} per quarter to about 5.2×10^{-5} per quarter. This is equivalent to using double copy by using just 1/5 of redundancy.

$$\text{RAID5 (5+1)} = 91/20 * 0.369\% * 0.308\% = 0.0052\% = 5.2 \times 10^{-5} / \text{trimester}$$

By extending the redundancy to two parity disks, RAID5 (5+2), using a Reed–Solomon scheme with two redundant symbols, the system can tolerate two simultaneous failures. In this case, the probability of an unrecoverable failure drops dramatically to approximately 2.2×10^{-7} per quarter.

$$\text{RAID5 (5+2)} = 91/20 * 0.431\% * 0.369\% * 0.308\% = 0.00000022\% = 2.2 \times 10^{-7} / \text{trimester}$$

Notice that a RAID5 (10+2) configuration has the same redundancy overhead as a RAID5 (5+1) system, while reducing the probability of unrecoverable failure by more than one order of magnitude. This improvement comes at the cost of increased computational complexity, as the encoding and recovery processes involve more advanced operations than simple XOR-based parity. This is three times smaller than using double copy by using the same 1/5 of redundancy.

$$\text{RAID5 (10+2)} = 91/20 * 0,738\% * 0,677\% * 0,615\% = 0.00000031\% = 3.1 \times 10^{-7} / \text{trimester}$$

$$\text{RAID5 (15+3)} = 91/20 * 1,108\% * 1,046\% * 0,985\% * 0,923\% = 0.000000048\% = 4.8 \times 10^{-8} / \text{trimester}$$

These results illustrate the practical impact of structured redundancy: with a relatively small increase in storage overhead (and computational power), it is possible to achieve several orders of magnitude improvement in reliability, making large-scale storage systems both efficient and robust.

3.4 – Transmission over high bit-flip error channels and retransmission

In channels with a high probability of random bit flips, the use of single-error-correcting codes such as the Hamming codes becomes increasingly unreliable. For example, with a bit-flip probability of 10%, the probability that a 72-bit packet contains more than one error is approximately 99.999%, making correct decoding highly unlikely.

One approach is to reduce the block size to the minimum, for instance 4 data bits plus 4 redundancy bits (extended Hamming). In this case, the probability that a packet is received without any error decreases to:

$$P(\text{no error}) = (1-0.1)^8 = 0.98 \approx 0.43$$

In bidirectional channels, such as the Internet, the availability of retransmission mechanisms changes the design strategy. With the possibility of repetition, it is no longer necessary to attempt correction at the receiver, which may lead to miscorrections when the number of bit errors is odd and greater than one. Then, about 43% of the packets are received error-free and can be immediately accepted, while, whenever a non-zero syndrome is detected (or a parity inconsistency is observed), it is immediately request retransmission, *i.e.*, there will be an automatic repeat request (ARQ) mechanism.

For a noise level of 10%, this strategy requires, on average, about 2.3 transmissions per successfully received packet, reducing the effective throughput to about 43.5%. Although this may seem inefficient, this reduction is not merely a consequence of the chosen strategy but reflects a fundamental limit of the channel. In fact, the maximum achievable fraction of useful information has been formally quantified by the Shannon (1948) Theorem. In a binary symmetric channel with bit-flip probability p , it is given by:

$$C = 1-H(p), \text{ with } H(p) = -p \times \log_2(p) - (1-p) \times \log_2(1-p)$$

where C is the channel capacity in bits per transmitted bit.

$$(p, C) = (0.1\%, 98.9\%); (0.3\%, 97.1\%); (1\%, 91.9\%); (3\%, 80.6\%); (10\%, 53.1\%); (30\%, 11.9\%)$$

A similar strategy is commonly used in storage systems. For example, during data recording, the information is first written to the medium and then immediately read back and compared with the original data. If a discrepancy is detected, the system attempts to rewrite the data and verifies it again. If the verification fails

repeatedly, the corresponding location is marked as a bad sector, and the write operation is redirected to a different physical area of the disk. This approach mirrors the use of retransmission in communication channels: rather than risking incorrect correction, the system relies on detection and repetition to ensure data integrity.

In systems combining Hamming codes and Reed-Solomon codes, an additional layer of robustness is obtained. Even if a miscorrection occurs at the Hamming level, the Reed-Solomon decoder can still detect inconsistencies at the symbol level.

If the Reed-Solomon code includes t redundant symbols, it can locate and correct up to $t/2$ symbols in error, or up to t erasures when the error locations are known. This means that some errors miscorrected at the bit level may still be identified and handled correctly at the higher level. Although this approach requires more computational effort, it provides a significant increase in reliability. In particular, it enables secure transmission and storage of sensitive information in scenarios where retransmission is not possible, such as in one-way communication channels or long-term data archiving.

As a concluding remark, if we expect digital information to survive for extremely long periods — on the order of thousands of years — and eventually hard drives become an archaeological artefact, redundancy is not optional but essential. Physical media degrade, bit errors accumulate, and no single layer of protection is sufficient. Only by combining multiple levels of redundancy, error detection, and correction (such as duplication, Hamming codes and Reed-Solomon codes), can long-term data integrity be realistically preserved.

3.5 – Non-classical methods

In addition to classical methods based on parity, Hamming codes and Reed-Solomon codes, modern error-correction algorithms have been developed that rely on more advanced approaches such as probabilistic inference and graph-based structures. Notable examples include LDPC and Turbo codes, which use iterative decoding techniques to approach the theoretical performance limit established by the Shannon Theorem. Compared to classical schemes, these methods achieve significantly higher efficiency, enabling transmission rates closer to the channel capacity and improved robustness in high-noise environments. However, this gain comes at the cost of increased computational complexity and a probabilistic decoding process, meaning that they do not provide absolute guarantees of error correction, but rather an extremely low probability of failure. Thus, while classical codes offer deterministic guarantees within well-defined limits, modern coding techniques prioritise near-optimal performance at the expense of complexity and strict certainty.

4. Conclusions

Reliable information storage and transmission require mechanisms capable of ensuring data integrity. At the hardware level, one straightforward approach is to eliminate errors by increasing physical robustness or by duplicating information. However, such solutions are extremely costly, either due to the need for high-quality components or because of the significant overhead associated with full data replication.

The development of error-correcting codes represents a fundamental breakthrough in this context. Instead of relying on brute-force redundancy, mathematical models allow the introduction of carefully structured redundancy, enabling the detection and correction of errors with a fraction of the overhead required by simple repetition. This leads to improvements of several orders of magnitude in efficiency, making reliable communication and storage feasible at scale.

Among these methods, Hamming codes provide an elegant and efficient solution for correcting single-bit errors within small data blocks, with minimal redundancy. For larger-scale reliability, particularly in the presence of burst errors or complete data loss, Reed–Solomon codes extend this principle by introducing redundancy across multiple symbols, allowing the recovery of missing or corrupted data.

In this work, Reed–Solomon coding has been presented using a simplified integer-based formulation. This approach prioritizes conceptual understanding over strict adherence to finite field arithmetic, making the underlying principles more accessible. In practical systems, however, implementations rely on finite fields to ensure bounded symbol sizes and computational efficiency.

Without these redundancy models, it would not be possible to achieve the storage densities observed in modern DRAM, HDDs, and SSDs, nor the transmission speeds that underpin today’s communication networks. The presence of noise and physical limitations would make such levels of performance unattainable without efficient error correction.

These techniques are widely used in modern technology, including disk controllers, memory systems, communication protocols, and optical media such as CDs. Their pervasive adoption highlights the importance of mathematically grounded redundancy schemes, which provide robust error correction while maintaining low overhead.

In summary, error-correcting codes demonstrate that reliability does not require excessive redundancy. Instead, through carefully designed mathematical structures, it is possible to achieve high levels of data integrity with remarkably efficient use of additional information.

5. References

Backblaze(2025), <https://www.backblaze.com/blog/backblaze-drive-stats-for-2025/>

Hamming, R. W. (1950), “Error detecting and error correcting codes”, *Bell System Technical Journal*, 29(2), 147–160.

Reed, I. S., & Solomon, G. (1960), “Polynomial codes over certain finite fields”, *Journal of the Society for Industrial and Applied Mathematics*, 8(2), 300–304.

Shannon, C. E. (1948), “A Mathematical Theory of Communication”, *Bell System Technical Journal*, 27(3), 379–423 and 27(4), 623–656.